Constrained Randomization and Coverage

Task#1: Generate constrained-random tests vectors for an ALU

Design.sv

```
module alu (
    input  logic [2:0] opcode,
    input  logic [7:0] a, b,
    output logic [7:0] result,
    output logic       zero,
    output logic       carry,
    output logic       overflow
);

    logic [8:0] sum, diff;   // 9-bit for ADD/SUB (carry + 8-bit result)

    always_comb begin
        result   = 8'h00;
        zero     = 1'b0;
        carry    = 1'b0;
        overflow = 1'b0;

        case (opcode)
            0: begin // ADD
                sum = {1'b0, a} + {1'b0, b};
                result = sum[7:0];
                carry  = sum[8];
                overflow = (a[7] & b[7] & ~result[7]) |
                           (~a[7] & ~b[7] & result[7]);
            end


            1: begin // SUB
                diff = {1'b0, a} - {1'b0, b};
                result = diff[7:0];
                carry  = diff[8];
                overflow = (a[7] & ~b[7] & ~result[7]) |
                           (~a[7] & b[7] & result[7]);
            end

            2: begin // MUL
                result = a * b;         // product up to 16 bits, but we keep lower 8
                carry  = 1'b0;
                overflow = 1'b0;
            end

            3: begin // DIV
                if (b == 0)
                    result = 8'hFF;     // error value
                else
                    result = a / b;
                carry  = 1'b0;
                overflow = 1'b0;
            end
```

```
            4: result = a & b;          // AND
            5: result = a | b;          // OR
            6: result = a ^ b;          // XOR
            7: result = ~a;             // NOT

            default: result = 8'h00;
        endcase

        zero = (result == 8'h00);
    end
endmodule
```

Tb.sv

```
// ALU Testbench - Constrained-Random Vectors
module tb_alu;

    logic [2:0] opcode;
    logic [7:0] a, b;
    logic [7:0] result;
    logic       zero, carry, overflow;

    alu dut (.*);
    // ---------- Transaction class ----------
    class alu_trans;
        typedef enum bit [2:0] {
            ADD = 0,
            SUB = 1,
            MUL = 2,
            DIV = 3,
            AND = 4,
            OR  = 5,
            XOR = 6,
            NOT = 7
        } op_e;

        rand op_e       opcode;
        rand bit [7:0] a, b;

        // Weighted distribution
        constraint opcode_dist {
            opcode dist {
                ADD := 25,
                SUB := 20,
                MUL := 15,
                DIV := 10,
                AND := 10,
                OR  := 8,
                XOR := 7,
                NOT := 5
            };
        }

        constraint mul_small {
            if (opcode == MUL) {
                a inside {[0:15]};
                b inside {[0:15]};
            }
        }

        // For DIV, avoid divide-by-zero
        constraint div_nonzero {
            if (opcode == DIV) b != 0;
        }
```

```
        // Soft constraint - prefer even numbers (can be
overridden)
        constraint soft_even {
            soft a % 2 == 0;
            soft b % 2 == 0;
        }

        function void display(string prefix = "");
            $display("%s opcode=%0d (%s), a=%0d, b=%0d",
                     prefix, opcode, opcode.name(), a, b);
        endfunction
    endclass

    // ---------- Reference model (for self-checking) ------
----
    function void compute_expected(
        input  logic [2:0] op,
        input  logic [7:0] a, b,
        output logic [7:0] exp_result,
        output logic       exp_zero,
        output logic       exp_carry,
        output logic       exp_overflow
    );
        logic [8:0] sum, diff;

        case (op)
            0: begin // ADD

                sum = {1'b0, a} + {1'b0, b};
                exp_result  = sum[7:0];
                exp_carry   = sum[8];
                exp_overflow = (a[7] & b[7] &
~exp_result[7]) |
                               (~a[7] & ~b[7] &
exp_result[7]);
            end
            1: begin // SUB
                diff = {1'b0, a} - {1'b0, b};
                exp_result  = diff[7:0];
                exp_carry   = diff[8];
                exp_overflow = (a[7] & ~b[7] &
~exp_result[7]) |
                               (~a[7] & b[7] &
exp_result[7]);
            end
            2: begin // MUL
                exp_result = a * b;
                exp_carry  = 1'b0;
                exp_overflow = 1'b0;
            end
            3: begin // DIV
                exp_result = (b == 0) ? 8'hFF : a / b;
                exp_carry  = 1'b0;
                exp_overflow = 1'b0;
```

```
            end
            4: begin // AND
                exp_result = a & b;
                exp_carry  = 1'b0;
                exp_overflow = 1'b0;
            end
            5: begin // OR
                exp_result = a | b;
                exp_carry  = 1'b0;
                exp_overflow = 1'b0;
            end
            6: begin // XOR
                exp_result = a ^ b;
                exp_carry  = 1'b0;
                exp_overflow = 1'b0;
            end
            7: begin // NOT
                exp_result = ~a;
                exp_carry  = 1'b0;
                exp_overflow = 1'b0;
            end
            default: begin
                exp_result   = 8'h00;
                exp_carry    = 1'b0;
                exp_overflow = 1'b0;
            end
        endcase

        exp_zero = (exp_result == 8'h00);
    endfunction
    // ---------- Test generation ----------
    initial begin
        // ***** DECLARATIONS FIRST (before any statements)
*****
        alu_trans trans;
        logic [7:0] exp_result;
        logic       exp_zero, exp_carry, exp_overflow;
        integer     pass_count = 0, fail_count = 0;

        // ***** NOW PROCEDURAL STATEMENTS *****
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_alu);

        trans = new();

        $display("\n===== Generating 15 Constrained-Random
ALU Vectors =====\n");

        repeat (15) begin
            if (!trans.randomize()) begin
                $error("Randomization failed!");
                $finish;
            end

            // Drive DUT
            opcode = trans.opcode;
            a      = trans.a;
            b      = trans.b;
            #1;
```

```
            // Optionally compute expected and compare
            compute_expected(opcode, a, b,
                             exp_result, exp_zero,
exp_carry, exp_overflow);

            trans.display("Input: ");
            $write("  DUT result=%3d, zero=%b, carry=%b,
overflow=%b",
                   result, zero, carry, overflow);
            $write("  Expected=%3d, zero=%b, carry=%b,
overflow=%b",
                   exp_result, exp_zero, exp_carry,
exp_overflow);

            if ((result === exp_result) &&
                (zero === exp_zero) &&
                (carry === exp_carry) &&
                (overflow === exp_overflow)) begin
                $display(" => PASS");
                pass_count++;
            end else begin
                $display(" => FAIL");
                fail_count++;
            end
            $display("----------------------------------------
-------------");
        end


$display("\n============================================");
        $display("SUMMARY: %0d PASS, %0d FAIL, Success:
%0.2f%%",
                 pass_count, fail_count,
                 (pass_count * 100.0) / (pass_count +
fail_count));

$display("============================================");
        $finish;
    end
endmodule
```

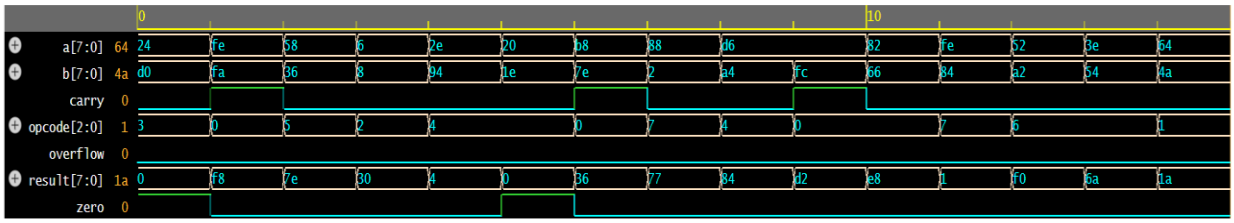
Output:

```
===== Generating 15 Constrained?Random ALU Vectors =====

Input:  opcode=3 (DIV), a=36, b=208
  DUT result=  0, zero=1, carry=0, overflow=0  Expected=  0, zero=1, carry=0, overflow=0 => PASS
--------------------------------------------------
Input:  opcode=0 (ADD), a=254, b=250
  DUT result=248, zero=0, carry=1, overflow=0  Expected=248, zero=0, carry=1, overflow=0 => PASS
--------------------------------------------------
Input:  opcode=5 (OR), a=88, b=54
  DUT result=126, zero=0, carry=0, overflow=0  Expected=126, zero=0, carry=0, overflow=0 => PASS
--------------------------------------------------
Input:  opcode=2 (MUL), a=6, b=8
  DUT result= 48, zero=0, carry=0, overflow=0  Expected= 48, zero=0, carry=0, overflow=0 => PASS
--------------------------------------------------
Input:  opcode=4 (AND), a=46, b=148
  DUT result=  4, zero=0, carry=0, overflow=0  Expected=  4, zero=0, carry=0, overflow=0 => PASS
--------------------------------------------------
Input:  opcode=4 (AND), a=32, b=30
  DUT result=  0, zero=1, carry=0, overflow=0  Expected=  0, zero=1, carry=0, overflow=0 => PASS
--------------------------------------------------
--------------------------------------------------
Input:  opcode=0 (ADD), a=184, b=126
  DUT result= 54, zero=0, carry=1, overflow=0  Expected= 54, zero=0, carry=1, overflow=0 => PASS
--------------------------------------------------
Input:  opcode=7 (NOT), a=136, b=2
  DUT result=119, zero=0, carry=0, overflow=0  Expected=119, zero=0, carry=0, overflow=0 => PASS
--------------------------------------------------
Input:  opcode=4 (AND), a=214, b=164
  DUT result=132, zero=0, carry=0, overflow=0  Expected=132, zero=0, carry=0, overflow=0 => PASS
--------------------------------------------------
Input:  opcode=0 (ADD), a=214, b=252
  DUT result=210, zero=0, carry=1, overflow=0  Expected=210, zero=0, carry=1, overflow=0 => PASS
--------------------------------------------------
Input:  opcode=0 (ADD), a=130, b=102
  DUT result=232, zero=0, carry=0, overflow=0  Expected=232, zero=0, carry=0, overflow=0 => PASS
--------------------------------------------------
Input:  opcode=7 (NOT), a=254, b=132
  DUT result=  1, zero=0, carry=0, overflow=0  Expected=  1, zero=0, carry=0, overflow=0 => PASS
--------------------------------------------------
Input:  opcode=6 (XOR), a=82, b=162
  DUT result=240, zero=0, carry=0, overflow=0  Expected=240, zero=0, carry=0, overflow=0 => PASS
--------------------------------------------------
--------------------------------------------------
Input:  opcode=6 (XOR), a=62, b=84
  DUT result=106, zero=0, carry=0, overflow=0  Expected=106, zero=0, carry=0, overflow=0 => PASS
--------------------------------------------------
Input:  opcode=1 (SUB), a=100, b=74
  DUT result= 26, zero=0, carry=0, overflow=0  Expected= 26, zero=0, carry=0, overflow=0 => PASS
--------------------------------------------------

==========================================
SUMMARY: 15 PASS, 0 FAIL, Success: 100.00%
==========================================
Simulation complete via $finish(1) at time 15 NS + 0
./testbench.sv:185         $finish;
xcelium> exit
TOOL:   xrun    25.03-s001: Exiting on Aug 23, 2026 at 12:47:43 EDT  (total: 00:00:02)
```

Waveform:



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.